Architecture Decision Record: ADR-001

Implementing OpenTelemetry Distributed Tracing

Abel-Raul Mazilu

13-05-2026

Status: Accepted
Project: DVerse Communication Platform 2.0

1 Context

The DVerse platform is evolving toward a decoupled multi-agent architecture in which AI agents operate as autonomous participants. These agents communicate asynchronously over Zenoh and also interact with users through the chat application interface.

As this architecture scales, the absence of distributed tracing becomes an engineering liability. Without end-to-end trace visibility, it is difficult to understand how a user request, agent action, Zenoh publication, message consumption, and downstream agent response are connected. Failures become difficult to isolate, latency sources remain opaque, and behavior across agent boundaries is not reproducible.

Sprint 4 must therefore introduce tracing at the same time as agent communication infrastructure. Retrofitting tracing later would likely produce incomplete coverage and inconsistent instrumentation.

2 Decision

Sprint 4 will implement OpenTelemetry as the distributed tracing foundation for the multi-agent system. Traces will capture agent lifecycle events, Zenoh message publication and consumption, chat interface interactions, and cross-agent workflows.

The implementation will establish trace spans for the following events:

- Agent startup, initialization, shutdown, and error states.
- Message publication to Zenoh topics.
- Message consumption from Zenoh subscriptions.

- Agent-to-agent request and response flows.
- Agent interactions that enter or leave the chat application.

Trace context must be propagated across Zenoh message boundaries. Because Zenoh does not provide HTTP-style headers for OpenTelemetry context propagation, the team will define a message-level propagation convention. This convention may include embedding trace context inside structured message metadata so that downstream agents can continue the same distributed trace.

3 Alternative Decisions

One alternative was to defer tracing to a later sprint. This was rejected because distributed tracing is most reliable when introduced alongside the communication paths it observes.

Another alternative was to use a vendor-specific tracing solution. This was rejected because OpenTelemetry provides an open standard, reduces vendor lock-in, and aligns with the broader cloud-native ecosystem.

4 Consequences

4.1 Positive

- Engineers will be able to follow a single interaction across agent boundaries, Zenoh topics, and chat application integration points.
- Latency analysis and error diagnosis will become easier as the number of agents and topics grows.
- Future observability tools can consume OpenTelemetry data without requiring a redesign of instrumentation points.

4.2 Negative

- Instrumentation adds implementation complexity during Sprint 4.
- Trace context propagation across Zenoh messages requires a documented convention.
- Overly granular tracing can introduce performance overhead in high-throughput message paths.

5 Scope

Deliverable	Description
Agent lifecycle tracing	Instrument startup, initialization, shutdown, and failure events.
Zenoh tracing	Trace publish and subscribe operations across Zenoh topics.
Chat tracing	Trace agent interactions entering or leaving the chat interface.
Trace propagation	Define and document message-level trace context propagation.

Statement on AI Use

Artificial intelligence was used to support the drafting of parts of this document. The generated text was subsequently reviewed, checked, and edited where necessary by me, the author, Abel-Raul Mazilu.